**UNIPAMPA — Universidade Federal do Pampa**
Engenharia de Computação

# RELATÓRIO PARCIAL

**Projeto Integrador V — Monitoramento de Nível de Reservatório/Barragem**

**Disciplina:** Projeto Integrador V
**Curso** Engenharia de Computação
**Turma** EC11
**Semestre** 2026/2
**Docentes:** Carlos Michel Betemps; Bruno Silveira Neves
**Tema do Projeto:** Sistema de Monitoramento de Nível de Reservatório/Barragem (IoT)
**Discentes:** Pedro B. Menezes; Rodrigo C. de Silveira; Rodrigo D. Peraça Jr.

## 1. Introdução

O presente relatório apresenta a atualização do desenvolvimento do Projeto Integrador V (PI5), cujo tema é um sistema de monitoramento de nível de água de reservatório ou barragem. A proposta integra sensoriamento local, comunicação em rede, serviços de backend e, em fases posteriores, informações meteorológicas e recursos de visualização. O estudo contempla tanto níveis elevados, associados à investigação de cheias, quanto níveis reduzidos, relacionados à análise de escassez hídrica.

Em continuidade ao primeiro relatório parcial, a equipe revisou decisões de arquitetura e reorganizou o desenvolvimento em tarefas progressivas. O envio inicialmente proposto por meio de ESP12F/Wi-Fi foi substituído, na arquitetura escolhida, por Ethernet Shield W5100; o backend preliminar em Python foi abandonado como implementação ativa e reiniciado em NestJS; e a estratégia de armazenamento passou de um banco relacional para MongoDB, com a utilização planejada do Prisma ORM.

O documento preserva a estrutura e o caráter de acompanhamento acadêmico do relatório anterior, distinguindo o desenvolvimento comprovadamente executado dos componentes em preparação. O sistema permanece em estágio de protótipo e não constitui, neste momento, equipamento operacional de alerta de risco em barragens.

## 2. Objetivos

### 2.1 Objetivo Geral

Desenvolver um protótipo IoT capaz de monitorar o comportamento do nível da água em uma maquete, registrar leituras identificadas temporalmente e disponibilizá-las por uma API, estabelecendo bases para o acompanhamento de níveis elevados e baixos e para análises futuras apoiadas por informações meteorológicas e séries históricas.

### 2.2 Objetivos Específicos

- Implementar e comparar duas formas de estimativa de nível: medição ultrassônica por HC-SR04 e medição resistiva após calibração, com uma referência física comum.
- Obter umidade relativa do ar por sensor DHT e data/hora da medição por módulo RTC.
- Utilizar Arduino Mega 2560 com Ethernet Shield W5100 para transmitir dados periodicamente em formato JSON por HTTP POST.
- Desenvolver API REST em NestJS/TypeScript com separação entre módulo, controller, service, DTO e persistência.

- Validar os dados recebidos, evitar duplicações e armazenar o histórico em MongoDB, com Prisma como camada de acesso planejada.
- Integrar dados meteorológicos da Open-Meteo e estudar séries históricas da ANA/HidroWeb para apoiar análises posteriores.
- Construir, em etapa futura, interface Next.js para consulta de estado, histórico e alertas.

## 3. Metodologia

O projeto segue a abordagem de Aprendizagem Baseada em Projetos (PBL) e práticas de organização ágil inspiradas em SCRUM, conforme descrito no relatório anterior. O backlog foi detalhado no Trello em tarefas numeradas, cada qual com uma entrega verificável e critérios de conclusão. A execução prioriza aprendizado incremental: primeiro contrato dos dados e comunicação HTTP, depois validação, banco de dados, comunicação física e demais integrações.

### 3.1 Gestão de atividades e responsabilidades

A organização da equipe continua baseada em Trello e rodízio de Scrum Master entre os três integrantes, seguindo a distribuição originalmente registrada: Incremento 1, Pedro Menezes; Incremento 2, Rodrigo Chaves; e Incremento 3, Rodrigo Peraça. O documento atual não atribui novas responsabilidades individuais nem indica a abertura formal de uma sprint adicional.

### 3.2 Estratégia de validação incremental

1. Preparar ferramentas de desenvolvimento e compilar um sketch mínimo para confirmar bibliotecas e seleção de placa.
2. Definir e versionar o contrato JSON V1 antes do desenvolvimento definitivo do firmware ou do DTO.
3. Criar o NestJS, testar POST com dados simulados pelo Bruno e armazenar temporariamente em memória.
4. Implementar validações de entrada, persistência e testes de idempotência.
5. Acoplar Ethernet Shield, sensores e módulos na maquete, validando comunicação real e calibração.

## 4. Arquitetura do Sistema

O fluxo adotado separa as funções de medição, transporte, recepção e armazenamento. Diferentemente da proposta inicial, o Arduino não se conecta diretamente ao MongoDB. Ele realiza uma requisição HTTP POST ao backend, que passa a ser responsável por validação, regras de negócio e persistência. Esse desacoplamento evita instalar credenciais de banco no microcontrolador e permite que o frontend consulte informações sem acessar a base diretamente.

| Camada | Componente e função |
|---|---|
| Sensoriamento | HC-SR04 (distância e nível calculado), sensor resistivo (nível após calibração), DHT (umidade) e RTC (horário). |
| Controle e transporte | Arduino Mega 2560 + W5100; firmware C++; Ethernet por cabo; HTTP POST com JSON. |
| Aplicação | NestJS: LeiturasController → LeiturasService → validações e operações de negócio. |
| Persistência futura | Prisma ORM → MongoDB: leituras, previsões meteorológicas e alertas. |
| Consulta futura | Next.js consome endpoints do NestJS para dashboard e histórico. |
| Fontes complementares | Open-Meteo para previsões e ANA/HidroWeb para pesquisa histórica em trilha separada. |

*Tabela 1 — Camadas e responsabilidades da arquitetura atual.*

### 4.1 Hardware e mudanças em relação ao relatório anterior

| Componente | Situação documentada | Observação |
|---|---|---|
| Arduino Mega 2560 | Plataforma definida; Arduino IDE configurado. | Montagem completa com W5100 ainda não validada. |
| HC-SR04 | Rotina de distância e média já estudada e testada anteriormente. | Exige calibração de referência para converter distância em nível. |
| Sensor resistivo vermelho | Selecionado como sensor complementar. | Depende de curva de calibração; sujeito à corrosão e à influência da água. |
| DHT e RTC | Componentes conhecidos dos ensaios anteriores. | A V1 prevê umidade e horário; temperatura não faz parte do contrato aprovado. |
| Ethernet Shield W5100 | Escolhido para substituir ESP12F/Wi-Fi. | Integração elétrica e conexão real ainda não demonstradas. |
| Buzzer e relé | Relatados como funcionais em ensaios prévios. | Não entram no payload V1; regras futuras de acionamento. |

### 4.2 Organização prevista do software

```
ProjetoIntegrador5/
  README.md
  docs/
    contrato-leituras.md
  backend/
    .env
    package.json
    src/
      main.ts
      app.module.ts
      leituras/
        leituras.module.ts
        leituras.controller.ts
        leituras.service.ts
        dto/create-leitura.dto.ts
        dto/update-leitura.dto.ts
        entities/leitura.entity.ts
  hardware/          # firmware C++ (organização prevista)
  frontend/          # Next.js (futuro)
  pesquisa-ana/      # notebooks/dados (frente paralela)
```

*Trecho de organização 1 — Estrutura-base do repositório (partes existentes e planejadas).*

O resource leituras foi gerado pelo Nest CLI com arquivos de módulo, controller, service, DTO e entidade. As pastas de firmware, frontend e pesquisa foram incluídas acima como destino de organização e não como prova de que já estão preenchidas no repositório.

## 5. Requisitos do Sistema

### 5.1 Aquisição e comunicação das leituras

- Identificar o dispositivo e cada medição, registrando o instante da coleta por RTC e, no servidor, o instante de recebimento.
- Enviar JSON V1 pelo endpoint POST /leituras, inicialmente a cada 5 segundos na maquete.
- Permitir que os demais sensores válidos continuem sendo reportados quando uma medição falhar.
- Considerar, como hipótese inicial de operação real, envios a cada hora; frequência definitiva dependerá de avaliação hidrológica e operacional.
- Obter confirmação HTTP de recebimento, implementar retentativas e preservar identificador de uma leitura retransmitida.

### 5.2 Contrato de dados V1

A especificação oficial foi documentada em docs/contrato-leituras.md. Os identificadores e a data/hora são obrigatórios. As quatro medições são opcionais individualmente, desde que ao menos uma esteja presente. Os níveis em centímetros só devem ser informados quando houver referência e calibração adequadas.

| Campo JSON | Tipo | Regra / origem |
|---|---|---|
| versao | inteiro | Obrigatório; igual a 1 na V1. |
| idDispositivo | string | Identificação estável do Arduino. |
| idLeitura | string | Identificador único da medição; preservado ao retransmitir. |
| timestampRtc | string ISO 8601 | Horário de coleta, com informação de fuso. |
| distanciaCm | número opcional | Distância HC-SR04 até superfície da água. |
| nivelUltrassonicoCm | número opcional | Nível obtido após definir a referência de altura. |
| nivelResistivoCm | número opcional | Nível do sensor resistivo somente após calibração. |
| umidadePct | número opcional | Umidade DHT na faixa de 0 a 100%. |

```
{
  "versao": 1,
  "idDispositivo": "arduino-mega-01",
  "idLeitura": "arduino-mega-01-20261022T160000-001",
  "timestampRtc": "2026-10-22T16:00:00-03:00",
  "distanciaCm": 35.7,
  "nivelUltrassonicoCm": 14.3,
  "nivelResistivoCm": 3.2,
  "umidadePct": 65.0
}
```

*Trecho de código 1 — Exemplo demonstrativo do JSON V1 (valores fictícios).*

### 5.3 Backend, persistência e segurança

- Aplicar validação de entrada com DTO e ValidationPipe; impedir campos não previstos e valores fisicamente inválidos.
- Persistir leituras no MongoDB com controle de duplicação por idDispositivo e idLeitura.
- Manter operações administrativas e credenciais de banco exclusivamente no backend; não permitir acesso direto do Arduino ao banco.
- Disponibilizar consultas por data, dispositivo e histórico por meio de endpoints REST.
- Na rede da maquete, testar por HTTP local; para instalação real, prever autenticação, proteção de transporte e controles de acesso.

### 5.4 Monitoramento, meteorologia e níveis críticos

O desenvolvimento futuro deverá atender a situações de nível elevado e nível baixo, definindo limites adequados à maquete e, em eventual aplicação real, aos parâmetros técnicos do reservatório. A previsão meteorológica poderá contextualizar a evolução do nível, mas não determinará sozinha o risco de cheia ou de desabastecimento. A atualização dos dados externos e sua persistência ainda não foram implementadas no NestJS.

## 6. Desafios Identificados

- Conectividade física e disponibilidade de rede: o W5100 exige cabo, rede alcançável e configuração correta de IP, porta e firewall.
- Qualidade metrológica: o ultrassônico mede distância, enquanto o sensor resistivo requer calibração própria; ambos devem utilizar a mesma referência para comparação.
- Durabilidade em contato com água: o módulo resistivo de trilhas expostas pode degradar e não é indicado, sem avaliação adicional, para monitoramento contínuo externo.

- Confiabilidade temporal: o RTC deve registrar horários consistentes, com regras claras de fuso e identificadores que não se repitam após reinicialização.
- Persistência e idempotência: retransmissões não podem produzir medições duplicadas; falhas no banco não devem ser mascaradas por respostas de sucesso.
- Escopo de risco: níveis críticos na maquete não equivalem automaticamente a transbordamento ou racionamento em instalações reais.
- Integração de versões e bibliotecas: Prisma deve ser instalado numa versão compatível com MongoDB; a compatibilidade deverá ser confirmada na tarefa correspondente.

## 7. Desenvolvimento Realizado até o Momento

### 7.1 Validação de hardware e ambiente

No primeiro relatório parcial, registrou-se a validação inicial do sensor ultrassônico em Tinkercad e em protoboard, incluindo medição por tempo de voo, média de cinco leituras e conversão da distância em nível por uma altura de referência. Esse resultado constitui histórico do projeto; não equivale à validação do conjunto final com Ethernet Shield W5100 e sensor resistivo.

Para a nova arquitetura, o Arduino IDE 2.3.10 foi configurado para Arduino Mega 2560. Um sketch contendo as bibliotecas SPI.h e Ethernet.h foi compilado sem erros, com indicação de 9.404 bytes de memória de programa (3%) e 461 bytes de memória dinâmica (5%). A compilação comprova disponibilidade do ambiente, mas não demonstra comunicação com a rede nem execução física do sketch.

### 7.2 Preparação de ferramentas e controle de versões

Foram verificados Node.js, npm, Git e VS Code em Windows/PowerShell. Versões inicialmente apresentadas: Node.js v22.16.0, npm 10.9.2, Git 2.45.1.windows.1 e VS Code 1.142.0. Após avisos EBADENGINE durante a instalação global do Nest CLI, o Node.js foi atualizado e o usuário informou que a nova configuração foi validada. A versão final do Node atualizada não consta das evidências registradas. Para testes HTTP, a equipe escolheu a extensão Bruno em substituição ao Thunder Client.

### 7.3 Backend NestJS e configuração inicial

O backend foi criado pelo Nest CLI com o gerenciador npm. O projeto dispõe de scripts para compilação e desenvolvimento, como npm run start:dev, e utiliza ConfigModule/ConfigService para carregar configurações de ambiente, sobretudo HOST e PORT. Durante a configuração ocorreram erros de porta inválida (NaN) e endereço formatado inadequadamente; a correção foi relatada e o servidor passou a iniciar normalmente.

```
// Exemplo do bootstrap de desenvolvimento
const config = app.get(ConfigService);
const port = Number(config.get<string>('PORT') ?? '3000');
const host = config.get<string>('HOST') ?? '127.0.0.1';
await app.listen(port, host);
```

*Trecho de código 2 — Inicialização do NestJS com host e porta configuráveis.*

O valor 127.0.0.1 permite acessos locais. Quando o Arduino estiver integrado, será necessário permitir escuta na interface de rede e utilizar no firmware o IP real do computador que hospeda o NestJS. O endereço 0.0.0.0 é opção de escuta do servidor, não endereço de destino para o Arduino.

### 7.4 Resource leituras e testes HTTP

O resource leituras foi criado por nest g resource leituras, com transporte REST API e métodos CRUD iniciais. A primeira implementação de POST adotou um array em memória no LeiturasService, usando Record<string, unknown> como tipo provisório até a criação do DTO definitivo. O método

recebe as propriedades do JSON e acrescenta o campo recebidoEm, que registra o horário do servidor.

```
@Injectable()
export class LeiturasService {
  private readonly leituras: Record<string, unknown>[] = [];

  create(dados: Record<string, unknown>) {
    const leitura = {
      ...dados,
      recebidoEm: new Date().toISOString(),
    };
    this.leituras.push(leitura);
    return { mensagem: 'Leitura recebida com sucesso!', leitura };
  }
}
```

*Trecho de código 3 — Persistência temporária em memória na prova de integração HTTP.*

Nos testes com Bruno, o POST /leituras retornou inicialmente 404 Not Found porque o LeiturasModule não havia sido importado no AppModule. Após o registro do módulo, o funcionamento do endpoint foi confirmado pelo desenvolvedor. Ainda falta registrar a confirmação do GET /leituras após alterar findAll() para retornar o array, pois a versão originalmente compartilhada ainda retornava uma string gerada pela CLI.

| Atividade | Situação na data de referência |
|---|---|
| T00.3 — Contrato V1 | Campos e política de envio definidos; arquivo produzido; versionamento no GitHub depende de confirmação. |
| T00.4 — Ambiente | Ferramentas validadas; Arduino IDE compilou sketch de teste. |
| T01.1 — Projeto NestJS | Criado e inicializado. |
| T01.2 — Ambiente e rede | ConfigModule, .env e execução local configurados. |
| T01.3 — Resource leituras | Estrutura e módulo gerados pelo CLI; módulo registrado. |
| T01.4 — POST mínimo | POST funcional em memória; GET ainda sem evidência final. |
| T01.5 — DTO real | Pendente: tipagem específica e validação global. |

### 7.5 Fontes de dados externas já estudadas

Open-Meteo. No backend antigo em Python foi realizado um GET de previsão horária com latitude e longitude próximas a Bagé/RS, forecast_days=3, timezone=America/Sao_Paulo e cinco variáveis: temperatura do ar, precipitação, probabilidade de precipitação, chuva e código do tempo. A prova de conceito retornou 72 previsões horárias. A implementação equivalente em NestJS, seu agendamento e armazenamento no MongoDB continuam pendentes.

ANA/HidroWeb. Foram obtidos CSVs das estações de Encantado/RS: 86720000_Cotas.csv, 86720000_Vazoes.csv, 2951010_Chuvas.csv e 86720000_QualAgua.csv. O objetivo é investigar a relação entre chuva antecedente, níveis altos e baixos e vazões, em estudo acadêmico separado da telemetria da maquete. O tratamento das séries e o treinamento de rede neural não integram entregas já comprovadas do novo backend.

### 7.6 Modelagem e funcionalidades ainda não implementadas

O MongoDB foi escolhido como solução futura de persistência, com Prisma ORM a ser configurado em versão compatível. Os modelos deverão representar o contrato V1 e incluir campos internos como identificador do documento e recebidoEm. O índice único previsto combina idDispositivo e idLeitura. Ainda não foram demonstradas conexão ao MongoDB, coleções persistidas, DTO validado, ingestão física via W5100, dashboard Next.js, regras de alertas ou processamento em FPGA.

## 8. Cronograma e Backlog — Próximas Entregas

O planejamento atual não fixa prazos de calendário nem durações para as tarefas. A sequência abaixo apresenta dependências e entregas técnicas a validar a partir do estágio atual.

| Ordem | Tarefa / objetivo | Critério principal de conclusão |
|---|---|---|
| 1 | Finalizar T01.4: consulta em memória. | GET /leituras retorna o array da sessão no Bruno. |
| 2 | T01.5: DTO V1 e validação global. | Casos válidos passam; faltas, tipos errados e faixas inválidas retornam 400. |
| 3 | Firmware inicial Ethernet W5100. | POST com JSON simulado chega à API pela rede local. |
| 4 | Montagem e calibração de sensores. | Distância, níveis comparáveis, umidade e RTC conferidos na maquete. |
| 5 | MongoDB + Prisma. | Leituras persistidas e deduplicadas por identificador. |
| 6 | Testes de robustez. | Retentativas, desconexão, validação e reinício documentados. |
| 7 | Open-Meteo no NestJS. | GET meteorológico e registros atualizados sem duplicações. |
| 8 | Next.js e alertas. | Dashboard consulta API; regras de níveis alto/baixo testadas. |
| 9 | ANA e modelo preditivo. | Análise e experimentos de rede neural documentados em frente própria. |

## 9. Próximos Passos

- Confirmar no Bruno o GET /leituras devolvendo os registros previamente enviados por POST na mesma execução.
- Implementar CreateLeituraDto com validações para os oito campos da V1 e adicionar ValidationPipe global com bloqueio de propriedades desconhecidas.
- Ajustar o controller e o service para receberem o tipo CreateLeituraDto, sem depender de Record<string, unknown>. Revisar endpoints CRUD desnecessários para telemetria.
- Criar sketch Ethernet com envio HTTP periódico e registro de resposta, inicialmente sem sensores físicos; testar conectividade em rede local.
- Montar o Ethernet Shield e os sensores da maquete, determinar altura de referência do ultrassônico e calibrar o sensor resistivo antes de expressar nível em centímetros.
- Configurar MongoDB e Prisma, documentar estratégia de índice único e confirmar persistência confiável.
- Planejar módulo Open-Meteo, regras de nível alto/baixo, visualização Next.js e estudo histórico ANA sem antecipar a etapa de FPGA.

## 10. Considerações Finais

O desenvolvimento do PI5 avançou da validação inicial do sensoriamento e da modelagem conceitual para uma implementação concreta de backend em NestJS, com contrato de comunicação V1 documentado e primeiro endpoint HTTP testado com dados simulados. A nova arquitetura privilegia uma divisão clara de responsabilidades: Arduino/Shield para aquisição e transporte, NestJS para processamento e validação, MongoDB para o histórico e Next.js para visualização futura.

A principal mudança técnica desde o primeiro relatório foi a reformulação da comunicação e do backend: a solução atualmente planejada utiliza Ethernet Shield W5100 e NestJS/TypeScript, mantendo a proposta de monitoramento de níveis elevados e baixos. O trabalho já realizado com Open-Meteo em Python e o levantamento de dados ANA permanecem como insumos de estudo, mas não devem ser confundidos com recursos implantados na nova aplicação.

O próximo marco verificável será concluir a rota de consulta em memória e formalizar a validação do JSON no CreateLeituraDto. A partir dessa base, a equipe poderá avaliar a transmissão por Ethernet e, posteriormente, a persistência dos dados. O relatório registra o estado do projeto em 23 de outubro de 2026, devendo ser atualizado à medida que novos testes sejam comprovados.

## Anexo A — Formato oficial de leitura aprovado

O exemplo abaixo é ilustrativo, não indica medições reais nem calibração concluída. As implementações futuras em C++, DTO e Prisma deverão respeitar essa estrutura e suas regras.

```
{
  "versao": 1,
  "idDispositivo": "arduino-mega-01",
  "idLeitura": "arduino-mega-01-20261022T160000-001",
  "timestampRtc": "2026-10-22T16:00:00-03:00",
  "distanciaCm": 35.7,
  "nivelUltrassonicoCm": 14.3,
  "nivelResistivoCm": 3.2,
  "umidadePct": 65.0
}
```

A política de retorno prevista distingue aceitação provisória (202), gravação efetiva (201), duplicação reconhecida sem nova gravação (200) e erro de entrada (400). Atualmente, o POST de demonstração segue o padrão 201 do NestJS mesmo sem persistir; o ajuste do status será avaliado na evolução da API.

## Anexo B — Fontes internas utilizadas na atualização


- **AGÊNCIA NACIONAL DE ÁGUAS E SANEAMENTO BÁSICO (ANA).** HidroWeb: Sistema Nacional de Informações sobre Recursos Hídricos. Disponível em: https://www.snirh.gov.br/hidroweb/.
- **ARDUINO.** Ethernet Library for Arduino. Documentação oficial da biblioteca Ethernet, compatível com o controlador W5100. Disponível em: https://github.com/arduino-libraries/Ethernet.
- **NESTJS.** First steps: documentação oficial do framework NestJS. Disponível em: https://docs.nestjs.com/first-steps.
- **NESTJS.** Controllers: documentação sobre controladores e rotas HTTP. Disponível em: https://docs.nestjs.com/controllers.
- **NESTJS.** Configuration: gerenciamento de variáveis de ambiente utilizando ConfigModule e ConfigService. Disponível em: https://docs.nestjs.com/techniques/configuration.
- **NESTJS.** Validation: validação de dados utilizando DTOs e ValidationPipe. Disponível em: https://docs.nestjs.com/techniques/validation.
- **NESTJS.** CLI usage: comandos para criação de projetos, módulos, controladores e serviços. Disponível em: https://docs.nestjs.com/cli/usages.
- **OPEN-METEO.** Weather Forecast API: documentação oficial da API de previsão meteorológica. Disponível em: https://open-meteo.com/en/docs.